

NEWSBOOK

Agrégateur d'actualités augmenté par IA :
vérification des faits, contexte historique et débat collaboratif

Rapport de projet : carnet de bord et démarche technique

Projet mené du 20 mai au 20 juillet 2026
Dépôt du code : github.com/Mac-Aurel/ohara

Sommaire

1. Introduction et problématique
2. Objectifs du projet
3. Architecture générale du système
4. Démarche méthodologique et choix techniques
 - 4.1 Le pipeline de scraping
 - 4.2 Résumé automatique
 - 4.3 Vérification des faits : itérations et compromis
 - 4.4 Regroupement des articles par histoire
 - 4.5 Authentification et débats threadés
 - 4.6 Recherche sémantique et génération augmentée (RAG)
 - 4.7 Modèle de données
 - 4.8 Déploiement
5. Carnet de bord chronologique
6. Étude de cas : diagnostic et correction d'un défaut de propagation du fact-check
7. Difficultés rencontrées et arbitrages
8. État actuel et résultats
9. Perspectives et travaux futurs
10. Conclusion

1. Introduction et problématique

La consommation d'actualités en ligne pose aujourd'hui un problème que la multiplication des sources n'a fait qu'aggraver. Un même événement est rapporté par des dizaines de médias, avec des angles, des degrés de vérification et des partis pris différents, sans qu'aucun outil grand public ne permette de croiser rapidement ces versions, d'évaluer la fiabilité d'une affirmation ou de replacer l'information dans un contexte historique plus large. Le lecteur est laissé seul face à un flux continu d'articles, avec pour seul repère la réputation de la source qu'il a choisi de lire.

Newsbook part de ce constat pour explorer une question précise : dans quelle mesure des modèles de langage, combinés à des sources vérifiables externes (Wikipedia, Open Library) et à des techniques de recherche d'information modernes (embeddings, recherche hybride), peuvent-ils outiller automatiquement un flux d'actualités pour en faciliter la lecture critique, sans pour autant se substituer au jugement du lecteur ? Le projet ne cherche pas à produire un « détecteur de fake news » au sens strict, mais un ensemble d'aides à la lecture : un résumé, une vérification factuelle claim par claim assortie d'un verdict nuancé, un contexte historique ancré dans des sources tierces, et un espace de discussion où les lecteurs peuvent eux-mêmes interroger le système sur les affirmations d'un article ou d'un commentaire.

Ce document restitue la démarche suivie sur environ deux mois de développement, du prototype initial jusqu'à l'état actuel du projet. Il présente les choix techniques opérés à chaque étape, les raisons qui les motivent, ainsi qu'un certain nombre de retours en arrière et de corrections qui, loin d'être des accidents de parcours, constituent une part importante du travail réellement effectué. Une étude de cas détaillée est notamment consacrée à un défaut de propagation des vérifications factuelles découvert et corrigé en fin de projet, qui illustre bien la différence entre un système qui fonctionne en apparence et un système dont le comportement a été vérifié empiriquement.

2. Objectifs du projet

Le projet s'est fixé un objectif fonctionnel central : construire un agrégateur d'actualités capable de collecter automatiquement des articles depuis plusieurs flux RSS, de les enrichir avec une vérification factuelle assistée par un modèle de langage, un contexte historique ancré sur des sources externes et des recommandations de lecture, puis de présenter ce contenu dans une interface web permettant la recherche, le filtrage, la discussion entre utilisateurs et l'interrogation directe du système sur le contenu des articles.

Au-delà de cet objectif fonctionnel, plusieurs contraintes techniques ont structuré les choix d'architecture tout au long du projet. La première est budgétaire : l'ensemble des services externes utilisés (modèle de langage, service d'embeddings, hébergement) devait rester dans des offres gratuites ou à très faible coût, ce qui a directement influencé le choix du fournisseur de modèle de langage et la cible de déploiement. La seconde est architecturale : le projet devait rester composé de services indépendants communiquant par HTTP, plutôt que d'un monolithe, afin de pouvoir faire évoluer, remplacer ou redéploier chaque brique séparément. La troisième, plus méthodologique, est la nécessité

de vérifier empiriquement chaque choix plutôt que de se fier à une intuition ou à une documentation générale : cette exigence transparaît en particulier dans le traitement du clustering d'articles, où plusieurs approches ont été testées, mesurées, puis acceptées ou rejetées sur la base de données réelles plutôt que d'un raisonnement théorique seul.

3. Architecture générale du système

Newsbook est structuré en neuf services indépendants, orchestrés par Docker Compose et communiquant entre eux par HTTP interne. Cette organisation en microservices a été retenue dès le départ pour deux raisons principales. D'une part, les différentes responsabilités du système (collecte de contenu, vérification factuelle, recherche sémantique, persistance) reposent sur des écosystèmes techniques différents : Node.js pour la couche de persistance et l'API orientée utilisateur, Python pour les traitements faisant intervenir des modèles de langage ou d'embeddings. D'autre part, cette séparation permet de faire évoluer un service sans redéployer l'ensemble du système, ce qui s'est révélé précieux lors des multiples itérations sur le service de vérification factuelle.

Le point d'entrée unique du système côté client est l'API Gateway, un service Express qui ne porte aucune logique métier et se contente de rediriger chaque famille de requêtes vers le service compétent. Cette centralisation simplifie la configuration du frontend, qui n'a besoin de connaître qu'une seule adresse, et fournit un point unique où appliquer des règles transverses comme la politique CORS.

Le cœur de la persistance est le news-service, un service Express adossé à une base PostgreSQL étendue par l'extension pgvector, qui permet de stocker des vecteurs d'embeddings directement dans les tables relationnelles et d'y effectuer des recherches par similarité cosinus sans dépendre d'une base vectorielle séparée. Ce choix évite d'introduire un second système de stockage pour un projet dont le volume de données reste modeste, au prix d'un léger surcoût de complexité dans les requêtes SQL. Autour de ce service gravitent le scraper, qui orchestre la collecte et l'enrichissement des articles, le summarizer et le fact-checker, deux services Python dédiés respectivement au résumé automatique et à la vérification factuelle, ainsi que le rag-service, responsable de la recherche sémantique et du chat conversationnel. Un service d'embeddings mutualisé (basé sur le serveur d'inférence Infinity) sert de point d'accès unique aux modèles de plongement vectoriel, afin d'éviter que chaque service Python ne charge sa propre copie du modèle en mémoire.

Architecture générale de Newsbook (microservices Docker Compose)

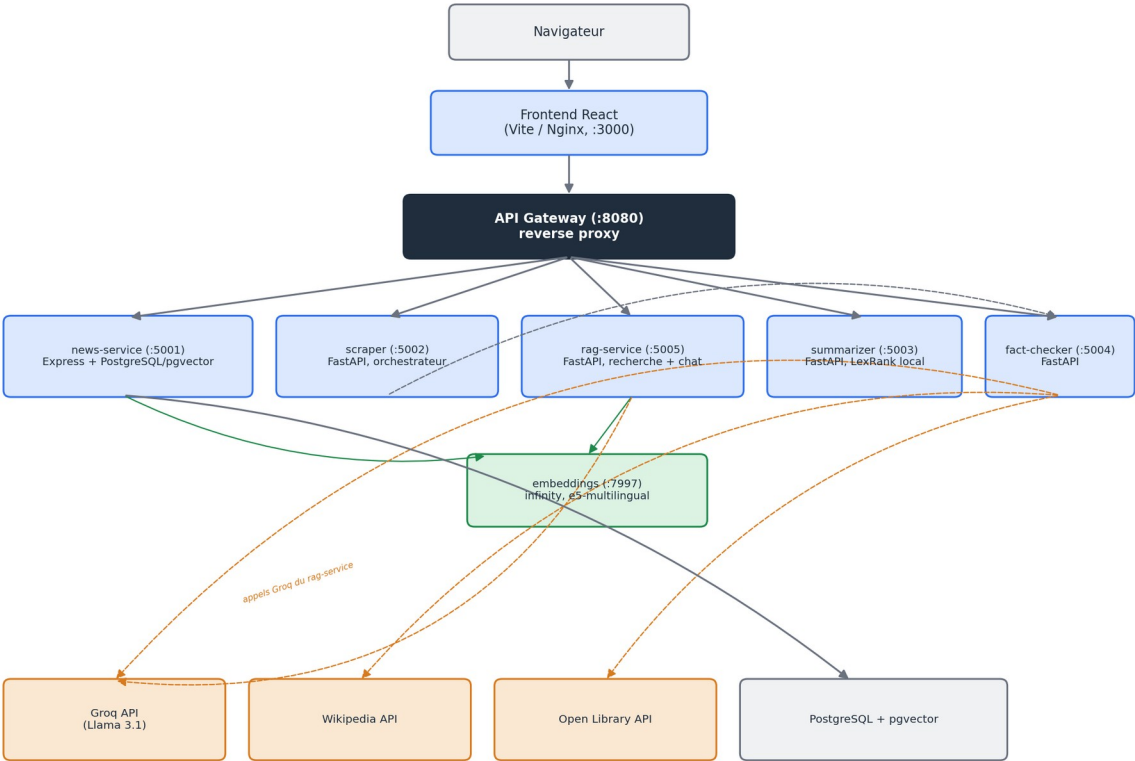


Figure 1 : architecture générale de Newsbook, microservices Docker Compose et leurs dépendances externes.

Cette architecture a évolué progressivement plutôt que d'avoir été figée dès le départ. Le service d'authentification, par exemple, était initialement prévu comme un service Node.js séparé dans la spécification d'origine ; il a finalement été intégré directement au news-service, la séparation en service distinct n'apportant pas de bénéfice réel tant que le nombre de routes concernées restait faible. Le même choix a été fait pour les débats threadés, initialement envisagés comme un service à part. Ces écarts par rapport aux spécifications initiales sont documentés explicitement dans les tickets correspondants du dépôt, avec la justification technique associée, plutôt que d'être passés sous silence.

4. Démarche méthodologique et choix techniques

Cette section détaille, service par service, les choix techniques opérés et les raisons qui les motivent. Elle suit l'ordre du pipeline de traitement d'un article, de sa collecte à son indexation pour la recherche sémantique, avant d'aborder les fonctionnalités transverses (authentification, débats) et le modèle de données qui les supporte.

4.1 Le pipeline de scraping

Le scraper est le point d'entrée de toute la chaîne de traitement. Il interroge quatre flux RSS (BBC, Reuters, The Guardian, Le Monde) en parallèle grâce à l'exécution de feedparser dans un pool de threads, feedparser étant une bibliothèque bloquante que l'on ne peut pas appeler directement dans une boucle asynco sans geler les autres tâches concurrentes.

Un choix de conception important a été de garder le scraper strictement sans état : il ne connaît aucune notion de planification et se contente de répondre à une requête POST /scrape lorsqu'on la lui envoie. Ce choix, discuté explicitement dans l'historique du projet, vise à permettre de faire tourner plusieurs instances du service derrière un répartiteur de charge sans qu'elles ne se déclenchent chacune de leur côté et ne dupliquent le travail. Le déclenchement périodique du scraping a d'ailleurs fait l'objet d'un aller-retour révélateur : une première implémentation avait réintroduit une boucle de planification interne au scraper pour simplifier le déploiement local, avant d'être explicitement identifiée comme une régression par rapport à ce principe et déplacée vers un conteneur dédié, minimaliste, qui ne fait qu'attendre puis appeler l'API Gateway à intervalle régulier. Ce conteneur agit comme un remplaçant local d'un service de cron externe, qui nécessiterait une URL publique non disponible tant que le projet n'est pas déployé.

Avant tout traitement coûteux, le scraper interroge le news-service pour connaître les URLs déjà présentes en base et les exclut du lot à traiter. Cette étape, ajoutée après qu'un défaut a été identifié en production, s'est révélée nécessaire pour deux raisons : elle évite de refaire, à chaque cycle de deux heures, le résumé et la vérification factuelle d'articles déjà traités, ce qui représenterait un gaspillage de quota d'appels au modèle de langage, et elle évite surtout d'écraser silencieusement une vérification factuelle déjà calculée par une nouvelle vérification recalculée dans un contexte différent. Pour les articles réellement nouveaux, le corps complet de la page est ensuite récupéré avec la bibliothèque trafilatura, le flux RSS ne fournissant généralement qu'un extrait tronqué, avec une image associée récupérée soit depuis le flux lui-même, soit depuis la balise og:image de la page HTML en dernier recours.

Pipeline de scraping : de la lecture des flux à l'indexation

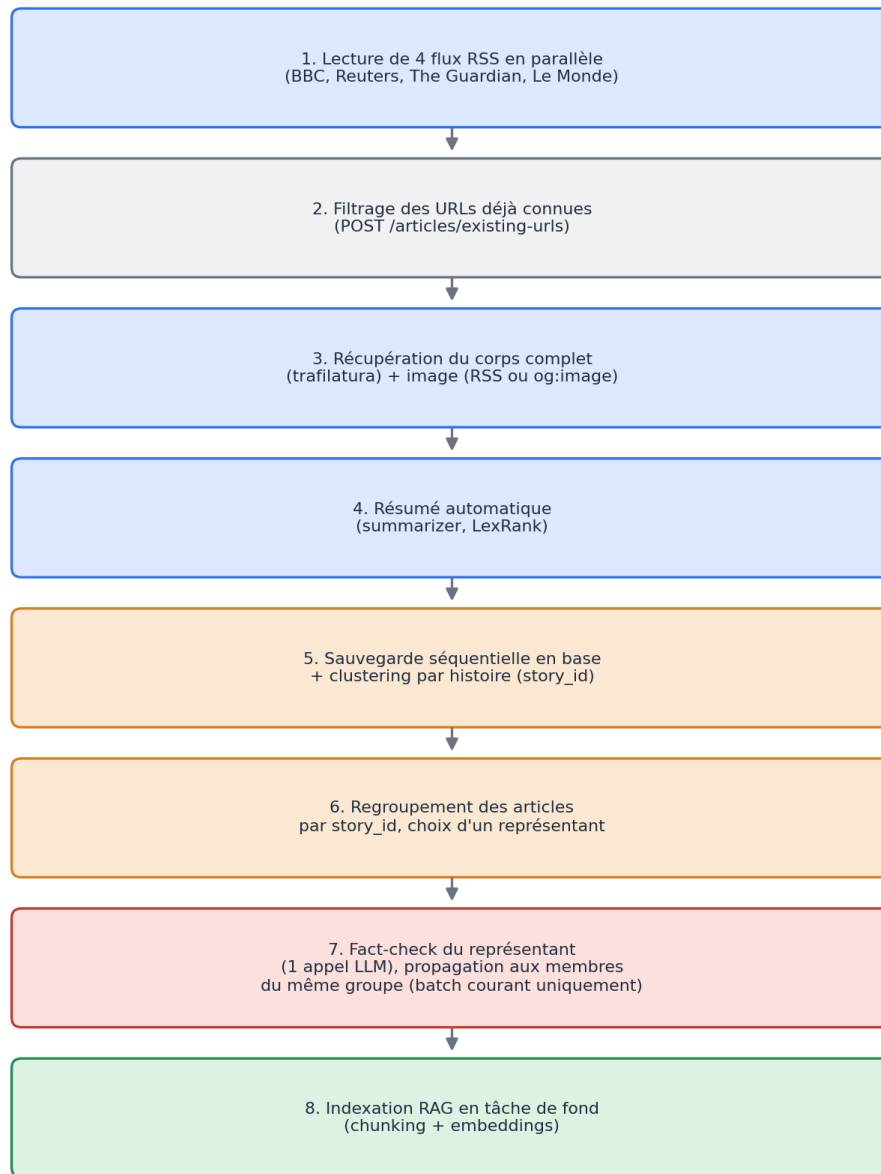


Figure 2 : pipeline de scraping, de la lecture des flux RSS à l'indexation pour la recherche sémantique.

4.2 Résumé automatique

Le service de résumé s'appuie sur l'algorithme extractif LexRank, via la bibliothèque sumy, exécuté localement sans appel à un modèle de langage externe. Ce choix a été fait pour des raisons de coût et de latence : un résumé extractif, qui sélectionne les phrases les plus représentatives d'un texte plutôt que d'en générer de nouvelles, est nettement moins coûteux en calcul qu'un appel à un grand modèle de langage, et ne consomme aucun quota d'API. La contrepartie est un résumé moins fluide qu'un résumé abstraktif généré par un LLM, un compromis jugé acceptable puisque le résumé n'est ici qu'un élément

secondaire de l'interface, l'essentiel de la valeur ajoutée du projet résidant dans la vérification factuelle et le contexte historique.

4.3 Vérification des faits : itérations et compromis

Le service de fact-checking est celui qui a connu le plus grand nombre d'itérations au cours du projet, et son historique illustre bien la nécessité d'arbitrer entre coût, débit et qualité lorsqu'on construit un système reposant sur un modèle de langage externe.

La première version s'appuyait sur l'API Groq et le modèle Llama 3.1 8B Instant, choisi pour la gratuité de son offre d'essai. Le service interroge Wikipedia pour retrouver des pages pertinentes à partir de mots-clés extraits du titre de l'article, injecte ces extraits dans le prompt envoyé au modèle avec pour consigne explicite de ne s'appuyer que sur ces sources pour le contexte historique, puis effectue un unique appel qui renvoie simultanément le verdict de vérification factuelle détaillé par affirmation, le contexte historique et une liste de recommandations de lecture. Ces dernières sont ensuite enrichies via l'API Open Library pour obtenir de véritables métadonnées bibliographiques plutôt que de faire confiance aveuglément aux références parfois approximatives suggérées par le modèle.

Ce choix initial s'est heurté à une limite concrète : l'offre gratuite de Groq plafonne à 6000 tokens par minute, ce qui, avec environ 3500 tokens consommés par article traité, ne permettait de traiter qu'un peu moins de deux articles par minute et allongeait un scraping complet à cinq voire neuf minutes. Une migration vers l'API Google Gemini Flash a alors été engagée, celle-ci annonçant un quota bien plus large, de l'ordre d'un million de tokens par minute contre six mille pour Groq, qui aurait permis de traiter l'ensemble des articles d'un cycle de scraping en parallèle, en une trentaine de secondes environ. Cette migration a cependant été abandonnée en cours de route : le compte utilisé renvoyait un quota nul sur l'offre gratuite de Gemini, un problème apparemment lié à une restriction régionale ou à la configuration du projet Google Cloud associé, et non reproductible de façon fiable.

Plutôt que de s'obstiner sur une piste bloquée pour des raisons externes au projet, le choix a été fait de revenir à Groq et de résoudre le problème de débit autrement, par optimisation de l'architecture logicielle plutôt que par changement de fournisseur : un unique appel au modèle par histoire plutôt que par article (le regroupement par histoire, décrit à la section suivante, sert justement à cela), combiné à l'activation du mode JSON strict de l'API, qui élimine les erreurs de parsing auparavant gérées par extraction de motifs dans le texte de réponse, à un prompt resserré, et à une parallélisation des appels à Wikipedia et Open Library. Ce choix illustre un principe suivi tout au long du projet : ne pas changer de fournisseur externe à la première difficulté rencontrée, mais d'abord chercher si le problème peut être résolu du côté du système que l'on maîtrise.

4.4 Regroupement des articles par histoire

Un même événement d'actualité est presque toujours couvert par plusieurs sources. Sans traitement particulier, l'interface afficherait autant de cartes distinctes que de sources ayant publié sur le sujet, ce qui noierait le fil d'actualité de doublons et empêcherait de ne vérifier factuellement chaque histoire qu'une seule fois plutôt qu'une fois par source.

La première implémentation reposait sur une similarité de Jaccard calculée sur les mots des titres, une méthode simple à mettre en œuvre mais qui ne fonctionnait de façon satisfaisante qu'en anglais : la comparaison lexicale brute échoue à rapprocher deux titres qui décrivent le même événement dans deux langues différentes, ce qui est pourtant précisément le cas d'usage le plus utile lorsque l'on agrège des sources francophones et anglophones comme Le Monde et la BBC. Ce constat a motivé le passage à une approche par embeddings sémantiques, testée d'abord de façon isolée avant d'être adoptée : le titre du nouvel article est transformé en vecteur par un modèle d'embeddings, comparé par similarité cosinus aux articles déjà en base, et rattaché à l'identifiant d'histoire (story_id) de l'article le plus proche si la similarité dépasse un certain seuil, ou associé à un nouvel identifiant sinon. Le modèle d'embeddings utilisé a lui-même dû être changé en cours de route : le premier modèle testé produisait des vecteurs peu discriminants pour les titres français, et a été remplacé par un modèle multilingue (multilingual-e5-small) capable de rapprocher correctement un titre français et son équivalent anglais.

Cette approche, bien que nettement supérieure à la comparaison lexicale, s'est révélée plus fragile qu'attendu lors d'un audit approfondi mené en toute fin de projet, qui a mis au jour un défaut de propagation des vérifications factuelles entre articles sans rapport. Ce défaut, sa cause, et la démarche de correction en deux temps qu'il a nécessitée font l'objet d'une étude de cas détaillée à la section 6 de ce document, qui constitue l'un des apports méthodologiques les plus significatifs de la dernière phase du projet.

4.5 Authentification et débats threadés

L'authentification repose sur un schéma classique nom d'utilisateur et mot de passe, les mots de passe étant hachés avec bcrypt avant stockage, et un jeton JWT signé avec un secret partagé (variable d'environnement JWT_SECRET) délivré à la connexion. Ce jeton est ensuite transmis dans l'en-tête Authorization de chaque requête nécessitant une identification, et vérifié par un middleware réutilisable côté serveur qui peut soit exiger une authentification valide, soit l'accepter de façon facultative lorsque la route se comporte différemment selon que l'utilisateur est connecté ou non, ce qui est le cas du fil d'actualité, qui indique par exemple si l'utilisateur courant a déjà aimé ou enregistré un article donné.

Les débats threadés permettent aux utilisateurs de commenter chaque article, avec des réponses imbriquées grâce à une clé auto-référencée parent_id sur la table comments, et une suppression en cascade lorsqu'un commentaire parent est supprimé. La fonctionnalité la plus originale de ce module est l'intégration d'un agent conversationnel accessible en mentionnant « @newsbook » dans un message : cette mention déclenche une requête vers le rag-service, restreinte à l'article courant avec repli sur une recherche globale si l'article n'est pas encore indexé, ou, si le message est une réponse à un autre commentaire, transmet ce commentaire comme une affirmation à vérifier plutôt que comme une simple question. La réponse générée est postée automatiquement dans le fil de discussion avec ses sources, sous un compte marqué comme robot (is_bot) pour rester visuellement distinguable des contributions humaines.

4.6 Recherche sémantique et génération augmentée (RAG)

Le rag-service assure deux fonctions distinctes mais complémentaires : une recherche sémantique libre sur l'ensemble du corpus, et un chat conversationnel capable de répondre à une question en citant ses sources. Lorsqu'un article est indexé, son contenu est découpé en fragments d'environ 450 tokens sans chevauchement (un chevauchement avait été envisagé mais n'a montré aucun bénéfice mesurable lors des essais, tout en augmentant le coût de l'indexation, et a donc été abandonné), chaque fragment étant ensuite transformé en vecteur et stocké dans une table dédiée, aux côtés d'une représentation textuelle indexée pour la recherche par mots-clés.

La recherche elle-même combine deux méthodes de score qui se complètent : une similarité vectorielle par embeddings, efficace pour retrouver un passage sémantiquement proche même en l'absence de recouvrement lexical exact, et une recherche par mots-clés classique reposant sur le type tsvector de PostgreSQL, plus fiable sur les noms propres et les termes rares que les embeddings ont parfois tendance à diluer. Les deux classements sont fusionnés par la méthode du Reciprocal Rank Fusion, qui combine des classements de nature différente sans nécessiter de normaliser leurs scores sur une échelle commune. Un reranking supplémentaire par modèle cross-encoder a été ajouté en fin de projet pour affiner l'ordre final des meilleurs extraits avant leur transmission au modèle de langage, un cross-encoder évaluant la pertinence d'une paire question/passage de façon plus précise qu'une simple similarité vectorielle, au prix d'un coût de calcul plus élevé, ce qui justifie de ne l'appliquer qu'à un petit nombre de candidats déjà présélectionnés plutôt qu'à l'ensemble du corpus.

Le chat, enfin, transmet au modèle de langage les meilleurs extraits retrouvés avec une consigne stricte : répondre uniquement à partir de ces extraits, citer ses sources par numéro, et répondre explicitement qu'il ne sait pas plutôt que d'improviser lorsque rien de pertinent n'a été retrouvé. Cette contrainte, systématique dans les prompts du projet, vise à limiter le risque d'hallucination inhérent aux modèles de langage génératifs, en les cantonnant à un rôle de synthèse de sources fournies plutôt que de génération libre.

Pipeline RAG : indexation et recherche/chat conversationnel

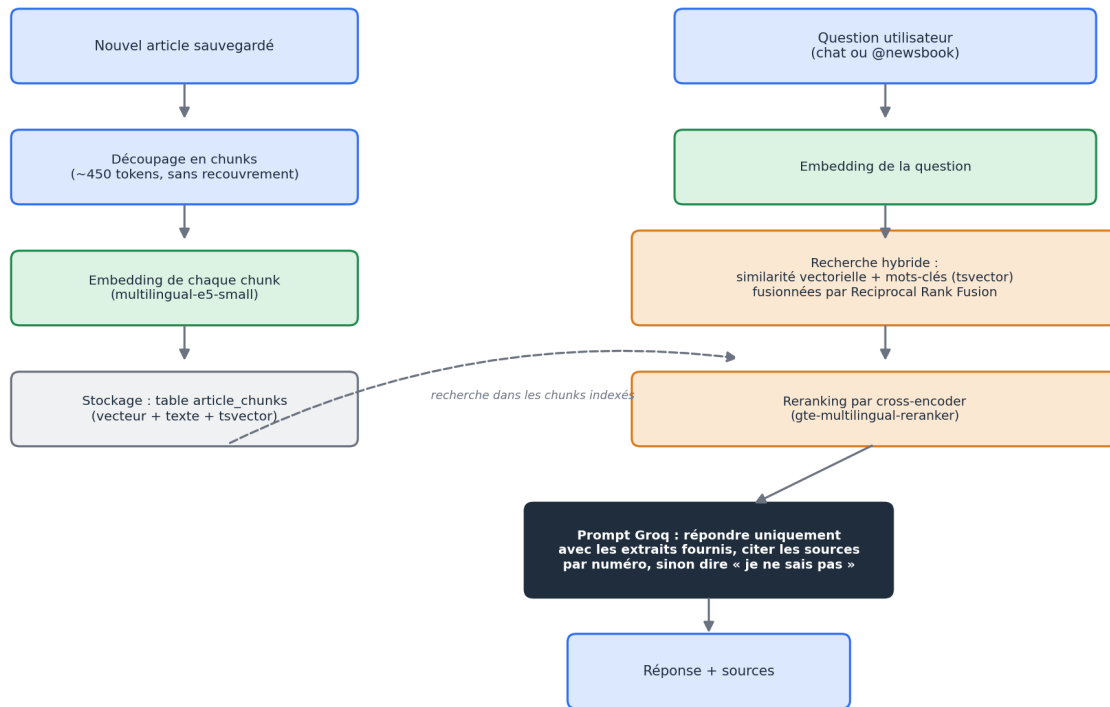


Figure 3 : pipeline RAG, indexation des articles et traitement d'une question utilisateur.

4.7 Modèle de données

L'ensemble du système repose sur une unique base PostgreSQL étendue par pgvector, ce qui permet de stocker à la fois les données relationnelles classiques (articles, commentaires, utilisateurs) et les vecteurs d'embeddings dans les mêmes tables, en évitant la complexité opérationnelle d'une base vectorielle dédiée pour un volume de données qui reste, à ce stade du projet, de l'ordre de quelques centaines d'articles. La table centrale articles porte à la fois le contenu de l'article, son embedding, son identifiant d'histoire (story_id) utilisé pour le regroupement décrit en section 4.4, et les résultats de l'enrichissement par le fact-checker, stockés au format JSONB pour conserver la structure détaillée par affirmation sans devoir créer une table supplémentaire.

Modèle de données simplifié (PostgreSQL + pgvector)

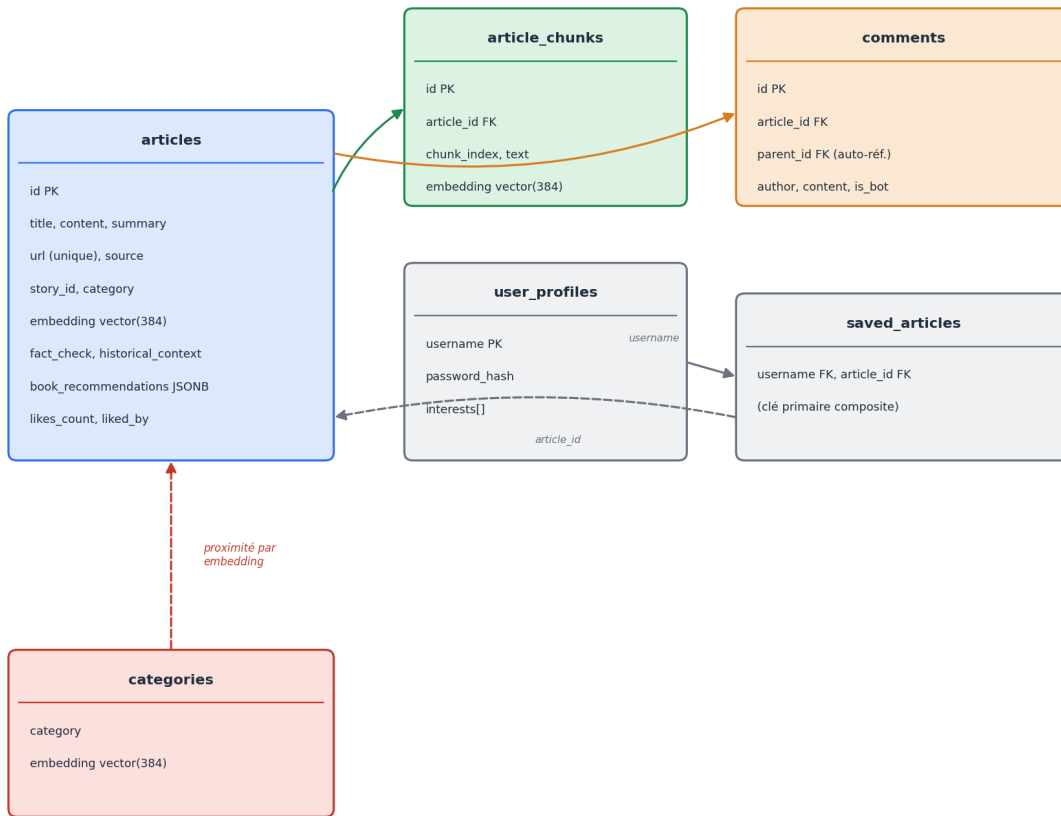


Figure 4 : modèle de données simplifié, tables principales et relations.

4.8 Déploiement

Le projet a d'abord tourné exclusivement en local via Docker Compose pendant le développement. La cible de déploiement envisagée dans les premières spécifications répartissait le système sur trois services distincts : Vercel pour le frontend, Neon pour une base PostgreSQL serverless, et cron-job.org pour le déclenchement périodique du scraping. Ce plan a été révisé en cours de projet au profit d'un déploiement sur une unique machine virtuelle ARM Oracle Cloud, dans le cadre de l'offre gratuite à vie proposée par ce fournisseur : la configuration matérielle disponible (quatre cœurs, vingt-quatre gigaoctets de mémoire) est largement suffisante pour héberger les neuf conteneurs du projet, y compris la base de données elle-même, ce qui rend inutile la complexité d'un déploiement multi-fournisseurs pour un projet de cette taille.

Cette bascule a nécessité l'introduction de Caddy comme serveur mandataire inverse en frontal, qui obtient et renouvelle automatiquement un certificat TLS via Let's Encrypt pour un sous-domaine DuckDNS, ainsi que le verrouillage explicite des ports de chaque service par la variable `BIND_ADDR` : en développement local, chaque service reste accessible sur toutes les interfaces réseau pour faciliter le débogage direct ; en production, cette même variable restreint chaque port à l'interface locale de la machine (127.0.0.1), de sorte que seul Caddy reste joignable depuis l'extérieur. La politique CORS de

l'API Gateway suit le même principe : ouverte par défaut en local, elle peut être restreinte au domaine public via la variable CORS_ORIGIN en production.

Architecture de déploiement — VM unique derrière Caddy (déployée)

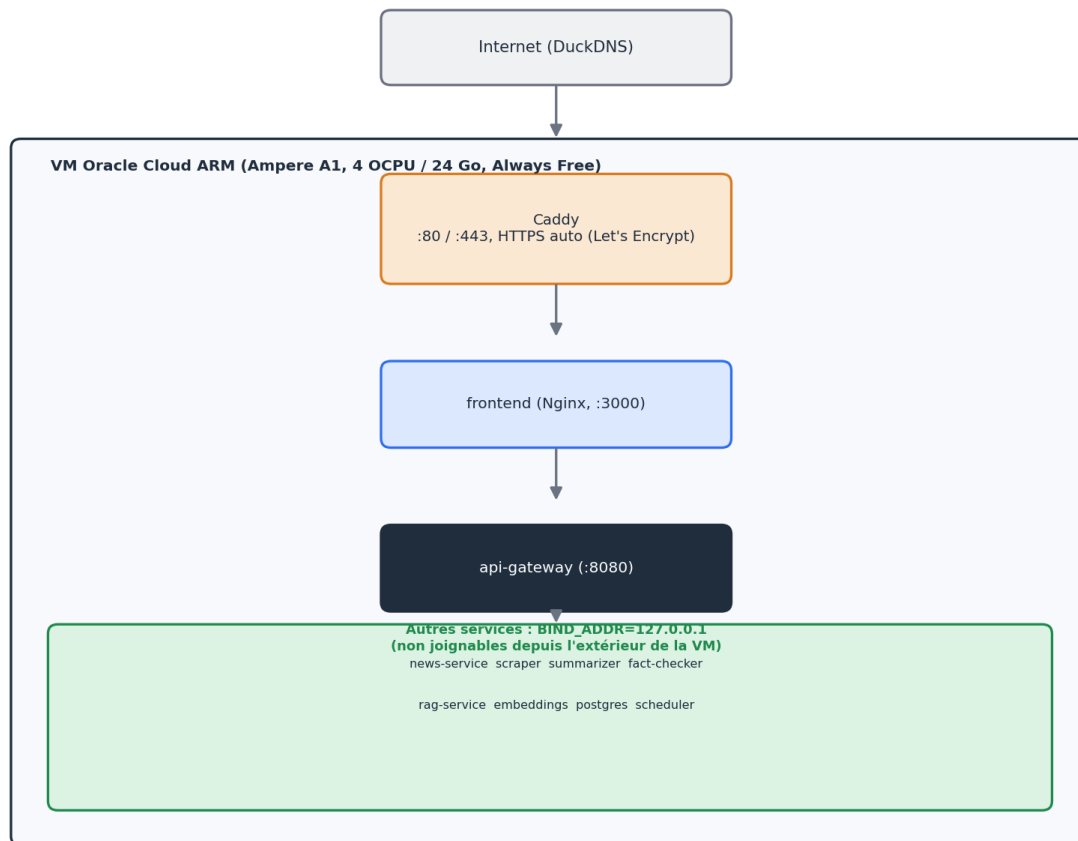


Figure 5 : architecture de déploiement, une unique machine virtuelle derrière un reverse proxy Caddy.

Cette configuration a depuis été déployée effectivement : la machine virtuelle ARM a été provisionnée sur Oracle Cloud, le sous-domaine DuckDNS pointé vers son adresse, et le projet est désormais accessible publiquement via Caddy, en HTTPS, suivant exactement l'architecture décrite ci-dessus. Le système ne tourne donc plus uniquement en local à ce stade du rapport.

5. Carnet de bord chronologique

Cette section retrace la chronologie effective du projet, telle qu'elle ressort de l'historique de développement, en mettant en évidence les jalons principaux et la logique qui a conduit de l'un à l'autre plutôt qu'un simple inventaire de fonctionnalités.

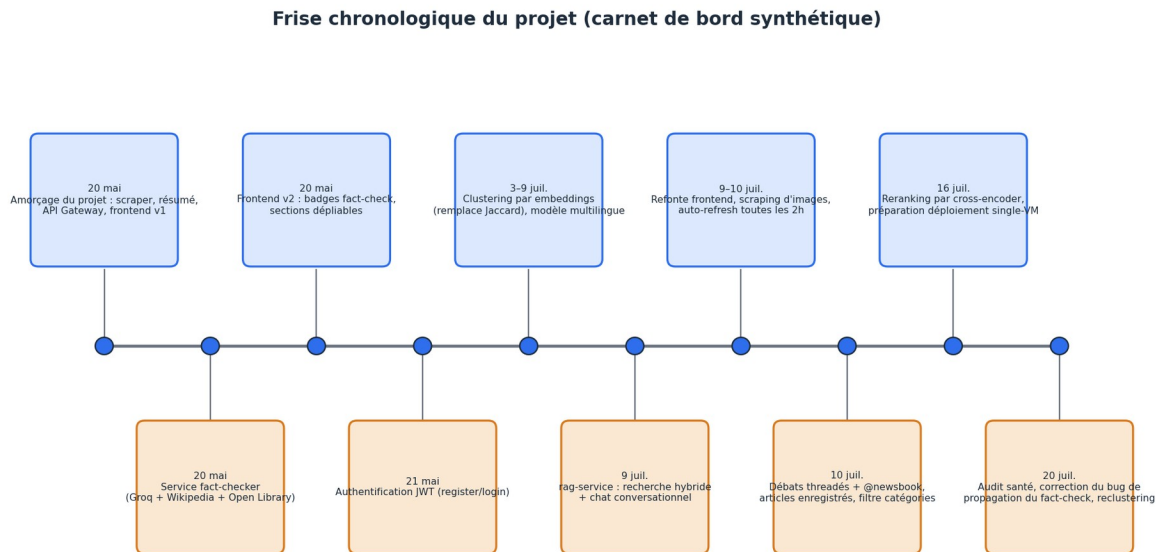


Figure 6 : frise chronologique synthétique du projet, du 20 mai au 20 juillet 2026.

Le projet démarre le 20 mai avec la mise en place du squelette applicatif : un scraper minimal, un service de résumé, une API Gateway et une première version du frontend. Cette même journée voit également la mise en place du service de fact-checking dans sa forme initiale, ainsi qu'une première itération du frontend ajoutant les badges de vérification factuelle et les sections dépliantes pour le contexte historique et les recommandations de lecture. Le 21 mai, l'authentification JWT est mise en place, ouvrant la voie aux fonctionnalités nécessitant d'identifier un utilisateur.

Après cette phase initiale intense, le projet connaît une pause avant de reprendre début juillet avec un chantier de fond : le remplacement du clustering par similarité de Jaccard par une approche fondée sur les embeddings sémantiques, accompagné d'un changement de modèle d'embeddings pour supporter correctement le français. Cette période voit également la mise en place d'une taxonomie de catégories fixes, après qu'une tentative de clustering automatique des catégories par l'algorithme HDBSCAN s'est révélée moins prévisible qu'une liste de catégories définies à l'avance et rapprochées des articles par similarité d'embedding.

Le 9 juillet marque l'introduction du rag-service et de la recherche hybride, rapidement suivie d'une refonte visuelle complète du frontend inspirée du site de référence raiseurvoice, de l'ajout de la récupération automatique d'images d'articles et d'un rafraîchissement automatique du flux toutes les deux heures. Le 10 juillet ajoute les débats threadés avec l'intégration de l'agent conversationnel, ainsi

que les fonctionnalités d'articles enregistrés et de filtre multi-catégories. Le 16 juillet introduit le reranking par cross-encoder dans le pipeline RAG et prépare la configuration de déploiement sur une unique machine virtuelle.

La dernière journée documentée dans ce rapport, le 20 juillet, correspond à un audit de santé du système en vue du déploiement, qui a mis au jour puis permis de corriger un défaut significatif de propagation des vérifications factuelles entre articles sans rapport. Cette dernière étape, suffisamment substantielle pour constituer un exercice de débogage à part entière, fait l'objet d'une étude de cas détaillée dans la section suivante.

6. Étude de cas : diagnostic et correction d'un défaut de propagation du fact-check

Cette section documente en détail un épisode de débogage survenu en fin de projet, à l'occasion d'un audit de santé du système mené avant le déploiement en production. Il est présenté ici de façon aussi complète que les autres choix techniques du projet, car il illustre un aspect essentiel de la démarche suivie : la vérification empirique d'un comportement, plutôt que la confiance aveugle dans une implémentation qui semble raisonnable sur le papier.

6.1 Symptôme observé

Le signalement initial est simple : un article consacré à l'arrivée de la tapisserie de Bayeux au British Museum affichait, dans l'interface, une section de vérification factuelle entièrement consacrée à une intoxication au méthanol survenue au Laos, sans aucun rapport avec le sujet de l'article. Ce type de symptôme, un contenu généré correct en apparence mais associé au mauvais article, est particulièrement insidieux : chaque composant du système, pris isolément, fonctionne normalement ; le défaut ne se manifeste que dans l'association entre deux éléments.

6.2 Hypothèse et vérification

L'hypothèse formulée dès le signalement portait sur le mécanisme de regroupement par histoire décrit en section 4.4 : la vérification factuelle n'est calculée qu'une fois par histoire, sur l'article le plus riche du groupe, puis propagée aux autres membres du même `story_id` par une requête de mise à jour de la forme `UPDATE articles SET fact_check = ... WHERE story_id = X`. Si deux articles sans rapport se voyaient attribuer le même `story_id` à la suite d'une erreur de regroupement, cette diffusion écraserait silencieusement la vérification factuelle correcte de l'un par celle de l'autre.

Cette hypothèse a été vérifiée directement sur les données en production plutôt que supposée vraie a priori. L'inspection de la base a confirmé qu'un article consacré à une polémique autour d'une bande dessinée du député François Ruffin partageait effectivement le même `story_id` que l'article sur la tapisserie de Bayeux, deux sujets Le Monde sans le moindre rapport thématique, et que l'article sur Ruffin affichait à son tour, au moment de l'inspection, la vérification factuelle du British Museum. La cause du symptôme signalé était confirmée, avec en prime la preuve que la contamination se déplaçait d'un cycle de scraping à l'autre, au gré des nouveaux faux positifs de regroupement.

6.3 Cause racine : un seuil de similarité mal calibré

L'origine du faux rapprochement a été recherchée du côté du calcul de similarité lui-même. Le regroupement par histoire comparait l'embedding du seul titre du nouvel article à celui de l'article existant le plus proche, et rattachait les deux au même `story_id` dès que la similarité cosinus dépassait 0,8. Un titre, pris seul, est un texte court et souvent générique, qui porte assez peu d'information distinctive : la mesure directe de la similarité entre les deux articles incriminés donnait 0,86, une valeur supérieure au seuil alors en vigueur, en dépit de l'absence totale de rapport entre les deux sujets.

Deux corrections complémentaires ont d'abord été apportées à ce stade du diagnostic : l'embedding utilisé pour le regroupement intègre désormais le résumé de l'article en plus de son titre, ce qui apporte davantage de signal distinctif, et le seuil de similarité a été relevé de 0,8 à 0,86. Un second changement, plus structurel, a également été apporté indépendamment du réglage du seuil : la propagation de la vérification factuelle a été restreinte aux seuls articles du lot de scraping courant, identifiés explicitement par leur identifiant, plutôt qu'à l'ensemble des lignes de la base partageant un story_id donné. Ce changement limite structurellement la portée d'un mauvais rapprochement : même si le regroupement se trompe à nouveau, il ne peut plus écraser la vérification factuelle d'un article ancien et sans rapport déjà présent en base.

6.4 Un premier correctif insuffisant : la comparaison au centroïde

Un examen plus systématique de l'ensemble de la base, mené par prudence après cette première correction, a révélé un problème plus profond que le cas isolé initialement signalé : plusieurs groupes d'articles comptant jusqu'à treize membres mélangeaient des sujets sans le moindre rapport entre eux, entre autres une épidémie d'Ebola, des feux de forêt sur trois continents différents, l'effondrement d'un pont à Gênes et des meurtres survenus dans une exploitation agricole au Kenya, tous regroupés sous un unique story_id. Le mécanisme en cause est un phénomène classique de chaînage propre aux méthodes de regroupement fondées sur la comparaison au plus proche voisin unique : un article A rejoint l'histoire d'un article B sur la base d'une similarité réelle et justifiée ; un article C, plus tard, rejoint à son tour cette même histoire sur la seule base de sa similarité avec A, sans jamais être comparé directement à B. Rien ne garantit alors que C ait le moindre rapport avec B, bien que les deux se retrouvent associés au même identifiant d'histoire, et donc à la même vérification factuelle.

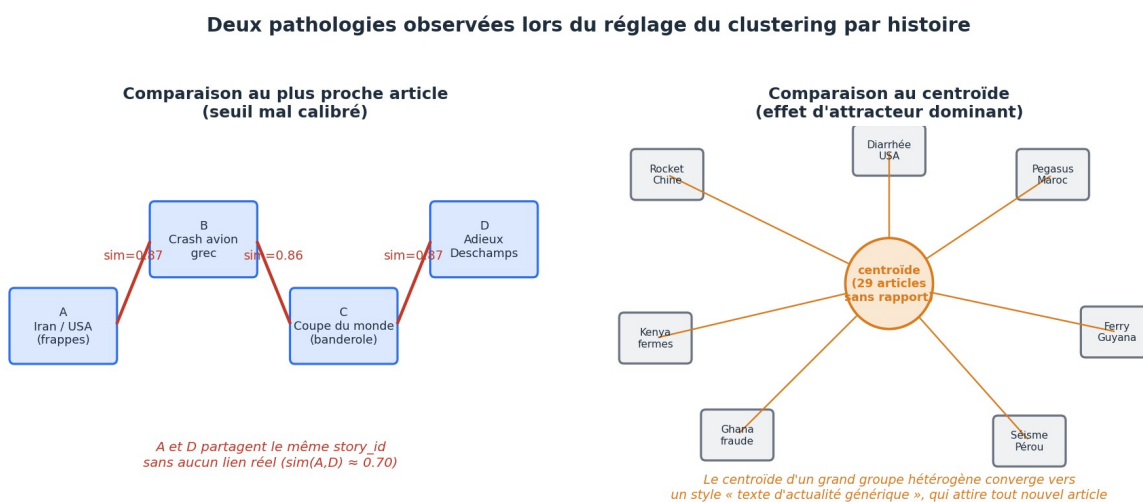


Figure 7 : les deux pathologies rencontrées lors du réglage du regroupement par histoire, chaînage transitif (à gauche) et effet d'attracteur dominant du centroïde (à droite).

Pour répondre à ce problème de chaînage, une première correction a consisté à comparer chaque nouvel article non plus à l'article existant le plus proche, mais au centroïde de chaque histoire, c'est-à-dire à la moyenne des embeddings de tous ses membres, l'intuition étant qu'un centroïde ne peut

dérivée que dans la mesure où ses membres le tirent réellement dans une direction donnée, contrairement à un unique article isolé qui peut, par hasard, se trouver proche de presque n'importe quoi. Cette correction a été mise en œuvre, déployée, puis testée directement sur les données réelles du projet plutôt que supposée fonctionner sur la seule base du raisonnement qui la motivait.

Le test a révélé que cette approche produisait un effet inverse à celui recherché, et nettement plus grave que le défaut qu'elle visait à corriger : un nouveau passage de traitement a produit un groupe de vingt-neuf articles anglophones sans le moindre rapport thématique entre eux : une attaque de missiles russes sur l'Ukraine, les adieux d'un sélectionneur de football, un séisme au Pérou, une fusée réutilisable chinoise, une affaire de fraude documentaire au Ghana, entre autres. L'explication tient à la nature même du centroïde d'un grand groupe hétérogène : à mesure qu'un groupe s'agrandit et couvre des sujets de plus en plus divers, son centroïde cesse de représenter un thème particulier et régresse vers une direction proche du style rédactionnel moyen d'une dépêche d'actualité en langue anglaise, une direction qui se trouve alors être raisonnablement proche de n'importe quel nouvel article de la même langue, quel qu'en soit le sujet. Le centroïde se comporte alors comme un attracteur qui absorbe indistinctement tout nouvel article venant à sa portée, un phénomène de rétroaction positive nettement plus destructeur que le chaînage progressif qu'il devait résoudre.

6.5 Solution retenue et validation empirique

Face à cet échec, l'approche par comparaison au plus proche voisin a été rétablie, mais avec un seuil recalibré non plus par intuition mais à partir de mesures directes effectuées sur le corpus réel du projet. Un ensemble de paires d'articles a été constitué pour cette mesure : des paires connues pour couvrir la même histoire sous deux formulations différentes (une même dépêche sur l'arrestation des frères Tate relayée en anglais et en français, un article sur le logiciel espion Pegasus au Maroc traité par deux sources), et des paires connues pour n'avoir aucun rapport entre elles, notamment tirées du groupe contaminé de vingt-neuf articles décrit plus haut.

Cette mesure a montré que les doublons authentiques se situaient entre 0,89 et 0,96 de similarité cosinus, tandis que les paires sans rapport pouvaient atteindre jusqu'à 0,84, laissant un intervalle relativement net entre les deux populations. Un seuil de 0,88 a été retenu, à l'intérieur de cet intervalle, comme valeur de compromis. Cette démarche, qui consiste à mesurer d'abord et à décider ensuite, contraste avec les deux réglages précédents (seuil de 0,8 puis de 0,86), qui reposaient sur un raisonnement plausible mais non vérifié contre des données réelles, et qui se sont tous deux révélés insuffisants à l'usage.

6.6 Nettoyage rétroactif du corpus

La correction de l'algorithme ne répare pas les regroupements déjà erronés présents en base. Un nettoyage rétroactif a donc été mené sur l'ensemble des articles appartenant à un groupe de plusieurs membres au moment de l'audit, soit près de la moitié du corpus total. La démarche a consisté à recalculer l'embedding de chaque article concerné avec le format à jour (titre et résumé), à réinitialiser son identifiant d'histoire, puis à rejouer l'algorithme de regroupement corrigé dans l'ordre chronologique d'insertion d'origine, de façon à reproduire fidèlement ce qu'aurait produit l'algorithme correct s'il avait

été utilisé depuis le début. Une vérification factuelle a ensuite été recalculée pour chaque groupe résultant de ce nouveau regroupement, sur la base de l'article le plus complet du groupe, puis diffusée aux autres membres de ce même groupe.

Ce nettoyage a lui-même dû être exécuté deux fois : une première passe, utilisant encore l'approche par centroïde, a reproduit à plus grande échelle le problème d'attracteur dominant décrit plus haut, avant d'être elle-même corrigée par une seconde passe utilisant l'algorithme définitif au seuil de 0,88. À l'issue de cette seconde passe, le plus grand groupe résiduel ne comptait plus que quatre membres, tous vérifiés manuellement comme couvrant effectivement la même histoire, et la similarité interne minimale de chaque groupe multi-articles restant s'établissait au-dessus de 0,86.

Cet épisode retient trois enseignements qui dépassent le cas particulier du regroupement d'articles. Premièrement, un correctif motivé par un raisonnement correct en apparence doit néanmoins être vérifié contre un comportement réel avant d'être considéré comme acquis : l'approche par centroïde était logiquement défendable et a pourtant aggravé le problème qu'elle visait à résoudre. Deuxièmement, un seuil de décision numérique gagne à être calibré sur des mesures effectuées sur les données du projet plutôt que choisi par approximation, une pratique d'autant plus importante que le coût d'un mauvais réglage (ici, une information factuellement fausse présentée comme vérifiée à l'utilisateur) est significatif. Troisièmement, une architecture qui limite la portée d'une erreur (la restriction de la diffusion du fact-check au lot de scraping courant, décrite en 6.3) reste une protection utile même lorsque l'algorithme en amont est correctement réglé, car aucun seuil ne peut garantir une exactitude parfaite sur des données aussi variées qu'un flux d'actualités.

7. Difficultés rencontrées et arbitrages

Au-delà de l'étude de cas détaillée à la section précédente, plusieurs autres difficultés ont jalonné le projet et méritent d'être restituées, car elles éclairent la façon dont les décisions ont été prises face à des contraintes imprévues.

La première tient aux limites de débit imposées par les offres gratuites de modèles de langage, déjà évoquées en section 4.3. Ces limites ont structuré une part importante des choix d'architecture du fact-checker et du rag-service : le regroupement d'articles par histoire n'a pas seulement une fonction d'affichage, il sert aussi directement à réduire le nombre d'appels nécessaires au modèle de langage, un unique appel par histoire remplaçant potentiellement plusieurs appels par article. De la même façon, le reranking par cross-encoder n'est appliqué qu'à un petit nombre de candidats déjà présélectionnés par la recherche hybride, plutôt qu'à l'ensemble du corpus, pour rester compatible avec un budget de calcul restreint.

La deuxième difficulté concerne le choix du modèle d'embeddings, revu à deux reprises au cours du projet : une première fois pour passer d'un modèle anglophone à un modèle multilingue capable de traiter correctement le français, une seconde fois, dans le cadre de l'étude de cas de la section 6, pour constater que même un modèle multilingue correctement choisi ne suffit pas à garantir un regroupement fiable si le texte fourni en entrée (le titre seul) est trop court pour porter un signal suffisamment distinctif, et si le seuil de décision n'est pas calibré empiriquement.

La troisième difficulté est plus organisationnelle que technique : plusieurs fonctionnalités (authentification, débats threadés, planification du scraping) ont été implémentées différemment de ce que prévoyait leur spécification initiale, généralement en simplifiant une architecture envisagée à l'origine comme plus distribuée qu'elle n'avait besoin de l'être à l'échelle du projet. Plutôt que de considérer ces écarts comme des manquements, le choix a été fait de les documenter explicitement dans chaque ticket concerné, avec la justification technique du changement, afin que la décision reste traçable et discutable plutôt que silencieuse.

8. État actuel et résultats

À l'issue des deux mois de développement couverts par ce rapport, Newsbook constitue un système fonctionnel de bout en bout, déployé publiquement sur une machine virtuelle Oracle Cloud suivant l'architecture décrite en section 4.8. Le système collecte automatiquement des articles depuis quatre flux RSS toutes les deux heures, les enrichit d'une vérification factuelle détaillée par affirmation, d'un contexte historique ancré sur des sources Wikipedia et de recommandations de lecture enrichies via Open Library, regroupe les articles couvrant la même histoire, et propose une interface permettant la recherche sémantique libre, un chat conversationnel citant ses sources, des débats threadés interrogeables par un agent automatique, l'enregistrement d'articles pour une lecture différée et un filtrage multi-catégories du fil d'actualité.

L'audit de santé mené en fin de projet, dont l'étude de cas de la section 6 constitue le développement principal, a permis de vérifier concrètement le comportement du système sur ses données réelles plutôt

que de se fier à son fonctionnement apparent, et a conduit à corriger un défaut de fiabilité significatif ainsi qu'à fiabiliser le service d'embeddings, qui ne redémarrait pas automatiquement après un arrêt inattendu faute de politique de redémarrage explicite dans sa configuration. L'ensemble des services du système répond désormais correctement à un contrôle de santé, et le corpus d'articles a été vérifié comme exempt de regroupements erronés au sens de la mesure de similarité interne décrite en section 6.6. Cet audit et la mise en production ont été menés de façon conjointe, dans cet ordre : le système n'a été exposé publiquement qu'une fois son comportement vérifié sur des données réelles.

9. Perspectives et travaux futurs

Le déploiement sur Oracle Cloud étant désormais effectif, le chantier restant le plus immédiat concerne le flux RSS de Reuters, qui ne renvoie plus aucune entrée exploitable : feedparser retourne systématiquement une liste vide pour ce flux, un problème qui semble tenir au flux lui-même (déplacé ou fermé côté Reuters) plutôt qu'à un défaut du côté du scraper, et qui nécessitera de rechercher une URL de remplacement ou une source alternative.

Le passage en production ouvre également la voie à une simplification déjà envisagée dès la conception du scraper : le déclenchement périodique du scraping repose pour l'instant sur un conteneur interne, minimaliste, qui joue le rôle d'un service de cron externe le temps que le projet ne dispose pas d'une URL publique. Cette contrainte n'existe plus une fois le système déployé, ce qui permettrait de basculer ce déclenchement vers un service de cron externe gratuit tel que cron-job.org, pointant directement sur l'adresse publique de l'API Gateway, comme cela avait été envisagé dans la spécification d'origine avant d'être reporté faute d'URL disponible.

Au-delà de ces deux chantiers, l'épisode documenté à la section 6 suggère une piste d'amélioration plus structurelle pour le regroupement d'articles : le seuil de similarité unique actuellement utilisé reste une heuristique, certes désormais calibrée sur des données réelles, mais qui ne garantit pas mathématiquement l'absence de faux positifs. Une évolution envisageable consisterait à valider un rapprochement non plus contre un seul élément (voisin le plus proche) ou une moyenne globale (centroïde), mais contre plusieurs membres du groupe candidat simultanément, ce qui limiterait à la fois le risque de chaînage et celui d'attracteur dominant identifiés dans cette étude, au prix d'un coût de calcul plus élevé à chaque insertion.

10. Conclusion

Ce projet a permis de construire, sur une période de deux mois, un agrégateur d'actualités augmenté par des modèles de langage, structuré en microservices indépendants et couvrant l'ensemble de la chaîne de traitement d'un article, de sa collecte à sa présentation enrichie dans une interface web désormais accessible publiquement. Au-delà du résultat fonctionnel, la démarche suivie tout au long du développement a privilégié la vérification empirique des choix techniques sur la confiance a priori : le changement de fournisseur de modèle de langage a été abandonné lorsque le problème initial a pu être résolu autrement, le modèle d'embeddings a été remplacé lorsque son comportement sur des titres français s'est révélé insuffisant, et surtout, l'algorithme de regroupement d'articles a fait l'objet, en fin de

projet, d'un cycle complet de diagnostic, de correction, d'échec d'une première solution pourtant motivée par un raisonnement solide, puis de correction définitive calibrée sur des mesures réelles plutôt que sur une intuition.

Cette dernière séquence, restituée en détail dans ce rapport, illustre à elle seule ce que la conduite d'un projet logiciel implique au-delà de l'écriture de code : la capacité à remettre en cause une solution qui semblait pourtant raisonnable, à la mesurer contre des données réelles plutôt que contre une intuition, et à documenter aussi bien les échecs intermédiaires que la solution finalement retenue. Le déploiement en production étant désormais acquis, le principal chantier restant, le remplacement du flux RSS de Reuters, constitue une suite naturelle et de portée nettement plus limitée que le travail déjà accompli.